Exploring Chess Conversations Across Reddit and 4chan: Data Collection System Implementation

Dev Desai
Binghamton University
Binghamton, NY, USA
ddesai4@binghamton.edu

Dev Nagi
Binghamton University
Binghamton, NY, USA
dnagi@binghamton.edu

***Abstract*—We implemented a continuously operating data collection system to gather chess-related discussions from Reddit and 4chan. Our system uses Faktory, a distributed job queue, to manage self-perpetuating data collection jobs that run 24/7 without manual intervention. The system successfully collected over 50,000 posts during development, demonstrating robust fault tolerance through automatic retry mechanisms and graceful handling of API rate limits. Data is stored in PostgreSQL using JSONB format for schema flexibility.**

I. Introduction

Chess has experienced significant growth in online communities, driven by platforms like Chess.com, Lichess, and popular streamers. This project implements a data collection system to capture chess discussions from Reddit and 4chan for future analysis of trends, sentiment, and event-driven activity spikes.

Our primary challenge was building a system that operates continuously without manual intervention while handling real-world issues including API rate limits, ephemeral content (4chan threads that disappear), and concurrent data collection from multiple sources.

II. System Architecture

A. Overview

Our system employs an event-driven architecture centered around Faktory, a distributed job queue system. Unlike time-based schedulers (such as cron), our approach uses self-perpetuating jobs where each task schedules its own future execution upon completion.

B. Core Components

Faktory Job Queue: Manages task scheduling, execution, and retry logic. Jobs are organized into four queues: crawl-thread-listing (4chan catalog monitoring), crawl-thread (4chan thread collection), crawl-subreddit (Reddit post collection), and crawl-reddit-comments (Reddit comment collection).

Worker Processes: Five concurrent workers execute jobs from the queues. Workers run continuously, requesting new tasks from Faktory upon completion of previous tasks.

API Clients: Custom HTTP clients for Reddit and 4chan APIs built using only the requests library. We access Reddit's public JSON endpoints which require no authentication beyond a User-Agent header. No pre-existing Reddit libraries such as PRAW were used.

PostgreSQL Database: Stores all collected data using JSONB format for schema flexibility. The database runs in a Docker container on port 5433.

C. Self-Perpetuating Job Pattern

Each collection job follows this pattern: (1) Execute data collection, (2) Store results in database, (3) Create dependent jobs, (4) Schedule next iteration of itself. This design ensures continuous operation after a single cold start, with jobs never overlapping.

III. Data Collection Strategies

A. Reddit Collection

We monitor three chess-related subreddits: r/chess, r/chessbeginners, and r/AnarchyChess. Every 10 minutes, jobs poll Reddit's public JSON endpoint to retrieve the 100 most recent posts. This endpoint requires no authentication—only a User-Agent header. For each new post, a separate job is created to collect all comments recursively.

To respect Reddit's rate limits (approximately 60 requests per minute), we implement 2-second delays between requests. When 429 rate limit errors occur, Faktory automatically retries the job with exponential backoff.

B. 4chan Collection

4chan threads are ephemeral—they disappear when they fall off the board catalog. Our strategy monitors the threads.json endpoint every 5 minutes. By comparing the current thread list with the previous iteration, we detect "dead threads." These threads are immediately crawled before they become permanently unavailable.

We collect all threads from /tg/ (Traditional Games) and /sp/ (Sports) boards rather than filtering during collection. This ensures comprehensive coverage regardless of terminology variations. Post-collection filtering will be applied during analysis.

IV. Implementation Changes from Proposal

A. Job Scheduling Architecture

Proposed: Cron-based scheduler running every 15 minutes.

Implemented: Faktory distributed job queue with self-perpetuating jobs.

Reason: Cron schedulers create race conditions when collection tasks exceed the scheduled interval. If a collection cycle takes 20 minutes, the next cron job starts while the previous is still running, causing database lock contention. Faktory's queue-based approach ensures jobs execute serially and provides automatic retry with exponential backoff.

B. 4chan Collection Scope

Proposed: Filter for chess-specific content during collection.

Implemented: Collect all threads from /tg/ and /sp/ boards.

Reason: Pre-filtering during collection risks missing relevant content due to terminology variations. Full board collection ensures comprehensive coverage, with post-collection filtering providing more accurate results using full post content.

C. Database Schema

Proposed: Not fully specified in proposal.

Implemented: TEXT fields for identifiers; JSONB for all post data.

Reason: Reddit uses alphanumeric identifiers (e.g., "1ocjrcz") while 4chan uses integers. TEXT fields accommodate both platforms. JSONB provides schema flexibility, preserving all API fields without pre-defining columns.

D. Rate Limit Handling

Proposed: Mentioned "backoff, 429/5xx."

Implemented: Multi-layered strategy combining 2-second delays, self-rescheduling on 429 errors, and Faktory's automatic retry with exponential backoff.

Reason: Reddit's rate limiting is complex and dynamic. Our approach balances collection throughput with API compliance. The system recovered within 1-2 hours from initial rate limits and maintained stable collection thereafter.

E. Worker Concurrency

Proposed: Not specified.

Implemented: 5 concurrent workers processing jobs from multiple queues.

Reason: Single-threaded collection creates bottlenecks. Five concurrent workers enable parallel processing, significantly improving throughput.

V. Challenges and Solutions

A. Reddit API Rate Limiting

Reddit enforces strict rate limits, returning 429 errors when limits are exceeded. Our initial burst of collection requests triggered these limits.

Solution: The self-scheduling job pattern naturally throttled request rates. When a job fails due to rate limiting, Faktory automatically reschedules it for 10 minutes later. All three subreddits recovered within 2 hours. This demonstrates fault tolerance—temporary API issues do not require manual intervention.

B. 4chan Thread Deletion

Approximately 5% of detected dead threads return 404 errors, indicating deletion by moderators or aging out of the archive.

Solution: Our system logs these as warnings and continues operation. Error handling uses graceful degradation to ensure individual thread failures do not impact overall system availability.

C. Database Schema Compatibility

Initial implementation used BIGINT for post identifiers, which failed when encountering Reddit's string-based IDs.

Solution: Modified schema to use TEXT fields, accommodating both platforms' identifier formats.

VI. Results

A. Collection Performance

During development, our system collected over 51,000 posts across all five sources. The system was subsequently redeployed to a university virtual machine for continued 24/7 operation.

Initial VM collection showed: 1,397 posts in 3 minutes (approximately 460 posts per minute). Sources active included r/chess (841 posts), r/chessbeginners (456 posts), and r/AnarchyChess (100 posts).

B. System Reliability

Faktory dashboard metrics showed: 9,044 successfully processed jobs, 17 failed jobs (0.2% failure rate), 0 permanently dead jobs. This 99.8% success rate demonstrates robust error handling with automatic recovery from all encountered errors.

C. Source Success Rates

All five data sources achieved successful continuous collection. Reddit sources encountered initial rate limiting but recovered within 2 hours through automatic retry. 4chan boards successfully collected posts from archived threads.

VII. Data Storage

Data is stored in PostgreSQL using JSONB format:

```
CREATE TABLE posts (
    id BIGSERIAL PRIMARY KEY,
    board_name TEXT NOT NULL,
    thread_number TEXT NOT NULL,
    post_number TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
    data JSONB NOT NULL,
    UNIQUE(board_name, post_number)
);
```

JSONB preserves all API fields, enabling flexible querying. Indexes on board_name, created_at, and thread_number ensure efficient queries. The UNIQUE constraint prevents duplicate data insertion, making the system idempotent.

VIII. Deployment

The system is deployed on a Binghamton University virtual machine using Docker containers. This enables 24/7 operation independent of personal computers, with reliable network connectivity and consistent computational resources.

IX. Conclusion

We successfully implemented a production-ready data collection system operating continuously without manual intervention. The system demonstrates fault tolerance, scalability, reliability, and observability. The system collected over 51,000 posts during development, proving capability to handle high-volume data collection from multiple platforms simultaneously.

Key architectural decisions—using Faktory instead of cron and implementing self-perpetuating jobs—proved essential for handling real-world challenges including rate limits, ephemeral content, and concurrent collection requirements. The deployment to a university virtual machine ensures continued operation, building a comprehensive dataset for future analysis.